Guía del profesor — Laboratorio Wi-Fi (Hacking Ético)

> **DOCUMENTO SOLO PARA EL PROFESOR.** Contiene las claves y la resolución completa de cada escenario. No lo repartas al alumnado: ellos tienen `guia_alumno.md`.

> **Entorno 100 % virtualizado.** Todas las radios son simuladas con `mac80211_hwsim` y aisladas en *network namespaces*. No hay emisión por radio real: las prácticas solo funcionan contra los APs de este laboratorio. **Uso exclusivamente educativo y en el aula.** Atacar redes reales sin autorización expresa por escrito es ilegal (en España, arts. 197 bis y 264 del Código Penal).

---

0. Puesta en marcha

```
sudo ./despliegue.sh
```

El script instala lo que falte (`hostapd`, `hcxtools`, `pocl-opencl-icd`, `ocl-icd-libopencl1`), levanta los 6 escenarios y, al terminar, imprime el **SSID, canal y BSSID real** de cada uno (las claves no se muestran). Los BSSID cambian en cada despliegue. Las interfaces de trabajo del alumno son **`wlan0` … `wlan6`**.

Para desmontarlo todo: `sudo ./limpia.sh`.

Poner una interfaz en modo monitor

```
sudo airmon-ng start wlan0 <canal>   # crea wlan0mon
# o, a mano:
sudo ip link set wlan0 down
sudo iw dev wlan0 set type monitor
sudo ip link set wlan0 up
```

No uses `airmon-ng check kill`. El script ya excluye las radios virtuales de NetworkManager, así que no hay nada que matar. Además, `ip netns exec` no aísla PIDs: `check kill` mataría los `wpa_supplicant` de los clientes del lab y los escenarios se quedarían sin tráfico (hay que volver a desplegar). `airmon-ng start` mostrará esos procesos como "could cause trouble": es solo un aviso y se puede ignorar.

Localizar objetivos y fijar canal

```
# Barrido general: anota BSSID y canal (CH)
sudo airodump-ng wlan0

# Fijar la interfaz al canal del objetivo (ej. canal 11)
sudo iw dev wlan0 set channel 11
```

Soluciones

SSID	Ch	Seguridad	Objetivo didáctico	Solución
`he_open`	1	Abierta	Sniffing de tráfico en claro (HTTP POST)	`user=admin&pass=SecretPass2026`
`he_wep`	9	WEP-104	Recolección de IVs + reinyección ARP	`ilovemyfamily` (hex `69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79`)
`he_wpa2`	6	WPA2-PSK	Captura del handshake 4-way + crackeo	`sunshine`
`he_pmkid`	11	WPA2-PSK	PMKID **con** clientes asociados	`butterfly`
`he_pmkid_nc`	4	WPA2-PSK	PMKID **sin** clientes (*clientless*)	`football`
`he_enterprise`	3	WPA2-EAP	Análisis pasivo de EAP-PEAP	Identidad `student_user@domain.lab` · tipo EAP 25 (PEAP) · cert. `radius.domain.lab`

Las tres claves WPA2 están en `rockyou.txt`. El password interno del cliente Enterprise (`EnterprisePass2026`) **no** es un objetivo de la práctica: el alumno solo debe identificar la identidad en claro y el método EAP.

---

1. `he_open` — Red abierta (tráfico en claro)

Un cliente envía un POST de login en bucle. El objetivo es demostrar que en una red sin cifrado las credenciales viajan legibles.

```
sudo iw dev wlan0 set channel 1
sudo airodump-ng -c 1 --bssid <BSSID_OPEN> -w cap_open wlan0
# En otra terminal, filtrar el POST en la captura:
tshark -r cap_open-01.cap -Y 'http.request.method == "POST"' -T fields -e
http.file_data
```

Resultado esperado: `user=admin&pass=SecretPass2026`.

Aprendizaje: cualquiera en la misma red abierta ve las credenciales.

Fundamento técnico

- **No hay cifrado en la capa de enlace.** En una red abierta el cuerpo de las tramas de datos 802.11 viaja tal cual: cabecera LLC/SNAP, IP, TCP y HTTP.
- **El medio es compartido.** Una interfaz en **modo monitor** entrega al sistema todas las tramas del canal, vayan dirigidas a ella o no (no filtra por `addr1`). No hace falta asociarse ni "atacar" nada: basta escuchar.
- **Lo que se explota es la ausencia de cifrado extremo a extremo.** El formulario se envía por HTTP, así que usuario y contraseña van legibles en el cuerpo del POST. Con HTTPS (TLS) se vería el tráfico, pero no su contenido.
- **Mitigación:** TLS en las aplicaciones (HTTPS, HSTS) y, en la red, **OWE / Wi-Fi Enhanced Open** (WPA3), que cifra cada cliente con una clave propia negociada por Diffie-Hellman sin necesidad de contraseña.

---

2. `he_wep` — WEP (IVs + reinyección ARP)

Dos clientes generan ARP broadcast en bucle, ideal para la reinyección. Basta una sola interfaz en monitor para capturar e inyectar.

```
# Monitor en el canal 9 (sin 'check kill')
sudo airmon-ng start wlan0 9

BSSID=<BSSID_WEP>

# 1. Capturar IVs (terminal 1, dejar corriendo)
sudo airodump-ng -c 9 --bssid $BSSID -w cap_wep wlan0mon
#    Anotar la MAC de un cliente de la lista STATION (p. ej. 02:00:00:00:13:00)
CLIENT=<MAC_CLIENTE>

# 2. Reinyección ARP suplantando al cliente (terminal 2)
sudo aireplay-ng -3 -b $BSSID -h $CLIENT wlan0mon
#    Si se reinicia, reutilizar el ARP ya capturado:
#    sudo aireplay-ng -3 -b $BSSID -h $CLIENT -r replay_arp-*.cap wlan0mon

# 3. Crackear en paralelo (terminal 3); reintenta solo cada 5000 IVs
aircrack-ng -b $BSSID cap_wep-*.cap
#    Si PTW no lo consigue: aircrack-ng -K -n 128 -b $BSSID cap_wep-*.cap
```

Resultado esperado: `KEY FOUND! [ 69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79 ] (ASCII: ilovemyfamily)` (con WEP-104, normalmente entre 20.000 y 80.000 IVs).

Fundamento técnico

Cómo cifra WEP cada paquete. WEP usa el cifrador de flujo **RC4**. La clave de cada paquete se construye pegando un **IV** (vector de inicialización, 24 bits) delante de la clave compartida:

```
clave RC4 del paquete = IV (24 bits, viaja EN CLARO) ‖ clave WEP (40 o 104 bits)
keystream             = RC4(IV ‖ clave WEP)
ICV                   = CRC32(datos)
cifrado               = (datos ‖ ICV) ⊕ keystream
trama                 = cabecera 802.11 ‖ IV ‖ KeyID ‖ cifrado
```

El receptor lee el IV de la cabecera, lo pega a la clave que ya conoce, regenera el mismo keystream y hace XOR para descifrar.

Qué se explota:

1. **Los primeros bytes de la clave RC4 de cada paquete son públicos.** El IV ocupa los 3 primeros bytes y viaja en claro. El algoritmo de inicialización de RC4 (KSA) tiene correlaciones estadísticas entre los bytes de la clave y los primeros bytes del keystream:
   - **FMS (2001):** solo los IV "débiles" filtraban información; hacían falta millones de paquetes.
   - **KoreK (2004):** más correlaciones; cientos de miles de IVs. Es el modo `-K` de aircrack-ng.
   - **PTW (2007):** basado en la correlación de Klein; sirve **cualquier** IV. Con WEP-104, ~40.000 paquetes dan un 50 % de éxito y ~85.000 un 95 %. Es el modo por defecto de aircrack-ng.
2. **Hay texto plano conocido para obtener el keystream.** El ataque necesita los primeros bytes del keystream de cada paquete, y keystream = cifrado ⊕ texto plano. Los paquetes ARP empiezan siempre igual: LLC/SNAP `AA AA 03 00 00 00 08 06` más la cabecera ARP `00 01 08 00 06 04 00 01`. Por eso PTW usa ARP: de cada uno se obtienen unos 16 bytes de keystream, cada paquete "vota" por el valor de cada byte de la clave y, con suficientes votos, la clave cae.
3. **No hay protección contra repetición.** WEP no lleva contador ni número de secuencia criptográfico: un paquete cifrado capturado es válido para siempre. El atacante reinyecta el mismo ARP una y otra vez. El AP lo acepta y lo retransmite en broadcast **cifrado con un IV nuevo**, y cada retransmisión es una muestra más para PTW. Esto es lo que hace `aireplay-ng -3`: pasar de unos pocos IVs por segundo a cientos.
4. **El IV es demasiado corto.** Con 24 bits solo hay 16,7 millones de IVs y, por la paradoja del cumpleaños, hay un 50 % de probabilidad de repetir uno a los ~5.000 paquetes. Mismo IV implica mismo keystream, y C1 ⊕ C2 = P1 ⊕ P2.
5. **La integridad es lineal y no lleva clave.** CRC32 no es un MAC: se pueden modificar bits del cifrado y recalcular el ICV sin conocer la clave (*bit-flipping*). Es la base de chopchop.
6. **La clave es estática y compartida.** Todos los clientes usan la misma clave, que no rota nunca.

Por qué no importa lo robusta que sea la clave. El ataque es estadístico, no de diccionario: no prueba contraseñas, sino que deduce cada byte de la clave a partir de las correlaciones. `ilovemyfamily` o 13 bytes aleatorios caen con el mismo número de IVs.

Fundamento del plan B (sin un ARP capturado):

- **Fragmentación (`-5`).** De cualquier paquete se conocen los 8 primeros bytes en claro (LLC/SNAP), así que se obtienen 8 bytes de keystream. 802.11 permite partir un paquete en hasta 16 fragmentos, y cada uno se puede cifrar con ese mismo keystream. El AP reensambla los fragmentos y retransmite el paquete completo con un IV nuevo. Como el atacante conoce lo que ha enviado, obtiene un keystream más largo, y repitiendo llega hasta ~1500 bytes.

- **Chopchop (`-4`).** Se quita el último byte de un paquete cifrado, se prueba cada uno de los 256 valores posibles y se corrige el ICV aprovechando la linealidad de CRC32. El AP descarta los ICV incorrectos y solo acepta el correcto, así que actúa de **oráculo**: se descifra el paquete byte a byte.
- **`packetforge-ng`.** Con un keystream y su IV se puede cifrar cualquier paquete: `(ARP ‖ CRC32(ARP)) ⊕ keystream`. Así se fabrica un ARP válido para reinyectar.

Mitigación: no hay ninguna dentro de WEP: está roto por diseño. Hay que usar WPA2 o WPA3.

Por qué `-h` con la MAC de un cliente y no fake auth (`-1`)

En un AP real se haría `aireplay-ng -1` (fake auth) y se inyectaría con la MAC propia. En `mac80211_hwsim` **no funciona**: el simulador solo genera ACK para interfaces activas, y una interfaz en monitor no lo es. El AP no recibe el ACK de la *Association Response*, hostapd no da por asociada la estación y descarta los ARP inyectados (aireplay muestra `got a deauth/disassoc packet`).

Síntoma típico: aireplay envía miles de paquetes, pero `#Data` en airodump apenas sube (2-3/s). Comprobación desde el host:

```
sudo ip netns exec ns_ap_wep iw dev wlan_ap_wep station dump | grep Station
# Solo aparecen los dos clientes reales, no la MAC de wlan0mon
```

Suplantando a un cliente ya asociado, el AP acepta los ARP y los retransmite con IVs nuevos: `#Data` sube a cientos por segundo.

La columna `Lost` de airodump

Con la reinyección, la estación suplantada muestra muchos `Lost`. No son paquetes perdidos: airodump calcula `Lost` por saltos en el número de secuencia, y el cliente real y aireplay emiten con la misma MAC y secuencias distintas. Se ignora; lo que importa es que `#Data` suba.

Plan B: sin ARP capturado (forjar uno)

Si `-3` no llega a capturar ningún ARP, se obtiene el keystream de cualquier paquete de datos y se forja un ARP:

```
# Keystream: fragmentación (si falla, chopchop con -4 -> replay_dec-*.xor)
sudo aireplay-ng -5 -b $BSSID -h $CLIENT wlan0mon          # -> fragment-*.xor

packetforge-ng -0 -a $BSSID -h $CLIENT -k 255.255.255.255 -l 255.255.255.255 \
               -y fragment-*.xor -w arp_forjado.cap

sudo aireplay-ng -2 -r arp_forjado.cap wlan0mon             # responder "y"
```

---

3. `he_wpa2` — Handshake 4-way + crackeo offline

Hay dos clientes asociados.

a) Capturar el handshake

```
sudo iw dev wlan0 set channel 6
sudo airodump-ng -c 6 --bssid <BSSID_WPA2> -w cap_wpa2 wlan0
```

b) Forzar reconexión con un deauth (desde otra terminal)

```
sudo iw dev wlan1 set type monitor && sudo ip link set wlan1 up
sudo iw dev wlan1 set channel 6
sudo aireplay-ng -0 3 -a <BSSID_WPA2> wlan1
```

Cuando airodump muestre `WPA handshake: <BSSID>` arriba a la derecha, ya lo tienes.

c) Crackear

```
# Con aircrack-ng
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID_WPA2> cap_wpa2-01.cap

# O con hashcat (modo 22000)
hcxpcapngtool -o hash_wpa2.22000 cap_wpa2-01.cap
hashcat -m 22000 hash_wpa2.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `sunshine`.

Fundamento técnico

Derivación de claves. La contraseña nunca viaja por la red. A partir de ella se deriva la **PMK**, y en cada conexión se deriva de la PMK una clave temporal, la **PTK**:

```
PMK = PBKDF2-HMAC-SHA1(passphrase, SSID, 4096 iteraciones, 256 bits)
PTK = PRF-512(PMK, "Pairwise key expansion",
              min(MAC_AP,MAC_STA) ‖ max(MAC_AP,MAC_STA) ‖
              min(ANonce,SNonce)  ‖ max(ANonce,SNonce))
KCK = primeros 128 bits de la PTK      (clave para el MIC)
```

Handshake 4-way:

Mensaje	Sentido	Contenido relevante
M1	AP → STA	ANonce (en claro)
M2	STA → AP	SNonce (en claro) + **MIC** = HMAC-SHA1(KCK, trama M2)
M3	AP → STA	GTK cifrada + MIC

Mensaje	Sentido	Contenido relevante
M4	STA → AP	Confirmación + MIC

Qué se explota:

- **Todo lo que entra en el cálculo, salvo la contraseña, viaja en claro:** SSID, MACs, ANonce, SNonce y la propia trama M2. El **MIC funciona como un verificador de la contraseña**. Para cada candidata del diccionario se calcula PMK → PTK → KCK → MIC; si coincide con el capturado, la contraseña es esa. Es un ataque **offline**: no hay que hablar con el AP ni hay límite de intentos. Basta con M1 + M2 (o M2 + M3).
- **Las tramas de gestión no van autenticadas.** Sin 802.11w (PMF), un deauth con la MAC del AP falsificada desconecta al cliente. Este se reconecta solo y repite el handshake delante del atacante. Por eso funciona `aireplay-ng -0`.
- **Contraste con WEP:** aquí no hay debilidad criptográfica, solo fuerza bruta. PBKDF2 con 4096 iteraciones hace cada intento caro. Con una contraseña larga y aleatoria el ataque es inviable; solo caen las contraseñas débiles o de diccionario (`sunshine` está en `rockyou.txt`).
- **El SSID actúa como sal:** las tablas precalculadas solo sirven para un SSID concreto. Por eso los SSID por defecto o muy comunes son más vulnerables.

Mitigación: contraseña larga y aleatoria; **WPA3-SAE** (el intercambio *Dragonfly* no deja material para un ataque de diccionario offline y aporta *forward secrecy*); **PMF (802.11w)** contra los deauth falsificados.

---

4. `he_pmkid` — PMKID **con** clientes

Hay una estación asociada generando tráfico. Sirve para contrastar las **dos vías**: capturar el PMKID (campo del M1) o el handshake 4-way tras un deauth.

Con hcxtools se consiguen las dos vías en una sola captura: hcxdumptool captura PMKID y handshakes a la vez.

a) Capturar con hcxdumptool (canal 11, solo este AP)

```
# Localizar el objetivo
sudo airodump-ng wlan0mon --band g       # he_pmkid: CH 11, con una estación
BSSID=<BSSID_PMKID>

# Filtro BPF para no atacar los otros escenarios
sudo hcxdumptool --bpfc="wlan addr3 $BSSID" > pmkid.bpf

# Interfaz limpia: hcxdumptool la pone en monitor él solo
sudo hcxdumptool -i wlan1 -c 11a --bpf=pmkid.bpf -w pmkid_cli.pcapng --rds=1
```

`pmkid.bpf` no forma parte del lab: lo genera `--bpfc` en la primera línea. Es el filtro compilado a bytecode BPF, en el mismo formato que `tcpdump -ddd`. `addr3` es el BSSID en las tramas de gestión y en el M1 del AP. El filtro es opcional, pero sin él hcxdumptool también ataca a los clientes de los otros escenarios del mismo canal.

hcxdumptool hace dos cosas a la vez:

- **Se asocia con el AP con una MAC inventada.** El AP le envía el M1 con el PMKID, igual que en `he_pmkid_nc`.
- **Ataca al cliente legítimo.** Lo desasocia y, cuando el cliente intenta reconectar, consigue su M2. Puede capturar el 4-way completo contra el AP real o hacerse pasar por el AP (pares *ROGUE*). En los dos casos el M2 lo calcula el cliente con la **clave real**, así que su MIC sirve para crackear. No es como el M2 de la contraseña falsa del método B de `he_pmkid_nc`.

En la pantalla salen **dos filas** para este AP:

- **Con la MAC de hcxdumptool** en `MAC-CL`: columna `P` marcada (PMKID).
- **Con la MAC del cliente real**: columnas `2` o `3` marcadas (handshake).

Cuando estén las dos, **Ctrl+C**. Bastan unos segundos o un minuto. No hay que dejarlo corriendo: mientras está activo sigue desconectando al cliente.

b) Convertir y separar los dos tipos de hash

```
hcxpcapngtool -o pmkid_cli.22000 pmkid_cli.pcapng

# Quedarse solo con he_pmkid (por si se ha colado otra red)
hcxhashtool -i pmkid_cli.22000 --essid=he_pmkid -o solo_pmkid.22000

grep -c '^WPA\*01' solo_pmkid.22000   # PMKID
grep -c '^WPA\*02' solo_pmkid.22000   # handshake (M1+M2)
```

En el resumen de hcxpcapngtool:

- `RSN PMKID written to 22000 hash file`: PMKID.
- `EAPOL pairs written to 22000 hash file`: handshakes.
- `EAPOL ROGUE pairs`: handshakes obtenidos haciéndose pasar por el AP.
- `MAC ACCESS POINT ... (incremented on every new client)`: el AP falso del propio hcxdumptool, no el del lab.

Cada línea 22000 tiene el formato `WPA*<tipo>*<PMKID o MIC>*<MAC_AP>*<MAC_STA>*<ESSID en hex>*...`.

c) Crackear

```
hashcat -m 22000 solo_pmkid.22000 /usr/share/wordlists/rockyou.txt
hashcat -m 22000 solo_pmkid.22000 --show
```

hashcat informa de `2 digests; 2 unique digests, 1 unique salts`: son dos hashes (PMKID y handshake) de la misma red, y el *salt* común es el ESSID. Los dos salen con la misma clave (`Recovered: 2/2`).

Para demostrar que cada vía funciona por separado:

```
hcxhashtool -i solo_pmkid.22000 --type=1 -o solo_pmkid_01.22000   # solo PMKID
hcxhashtool -i solo_pmkid.22000 --type=2 -o solo_pmkid_02.22000   # solo handshake
```

Resultado esperado: `butterfly`.

Problema frecuente: hashcat termina sin probar nada y muestra `Not enough allocatable device memory or free host memory for mapping`. La VM tiene poca RAM: PoCL solo deja reservar la mitad (por ejemplo `738/1477 MB`). Lo mejor es darle 4 GB a la VM. Si no es posible, reduce la carga:

```
hashcat -m 22000 solo_pmkid.22000 /usr/share/wordlists/rockyou.txt -n 1 -u 1 -T 1
-w 1
```

Si se prueba con un diccionario propio, comprueba que contiene la clave. hashcat no avisa: simplemente termina como `Exhausted`.

Fundamento técnico

- **Qué es el PMKID.** Es un identificador de la PMK pensado para el *roaming* y la caché de PMK (802.11i/r, OKC). Si el cliente vuelve a un AP que ya conoce su PMK, ambos la identifican por el PMKID y se ahorran repetir la autenticación completa. El AP lo incluye en el campo *Key Data* (RSN IE) del **M1**:

  ```
  PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)
  ```

- **Qué se explota.** Como el MIC del handshake, el PMKID es un **verificador de la contraseña** calculado con datos públicos (las dos MACs) y la PMK. Para cada candidata se calcula PMK → HMAC y se compara. El coste es el mismo que con el handshake (PBKDF2), pero basta **una sola trama (M1)**, enviada por el AP, en vez de M1 + M2. Lo descubrió Jens Steube (hashcat) en 2018.

- **Dos vías en este escenario.** Hay un cliente asociado, así que se puede conseguir el PMKID (asociándose uno mismo) o el handshake (deauth al cliente). hashcat las distingue en el formato 22000: `WPA*01` = PMKID, `WPA*02` = handshake.

  - Para el **PMKID** basta con el M1 del AP. Lo calcula el AP con la PMK real, sin importar quién se asocie.
  - Para el **handshake** hacen falta M1 y M2. El MIC del M2 solo vale como verificador si lo calcula alguien que conoce la clave, es decir, el cliente legítimo. Por eso esta vía necesita un cliente y la del PMKID no.

- **Ruido.** La vía del PMKID es sigilosa: solo hay una asociación del atacante. La del handshake obliga a desconectar al cliente, que lo nota como un corte breve y queda registrado en el AP.

- **Nota del lab:** el hostapd de serie no incluye el PMKID en el M1 con WPA-PSK. Por eso los escenarios PMKID usan un hostapd parcheado (`compila_hostapd_pmkid.sh`), que reproduce el comportamiento de muchos routers comerciales.

---

5. `he_pmkid_nc` — PMKID **sin** clientes (*clientless*)

No hay ninguna estación asociada **a propósito**. Es el caso que demuestra por qué el PMKID es peligroso: el AP incluye el PMKID en el **primer mensaje (M1)** del 4-way durante la asociación, así que **no hace falta ningún cliente**: basta con provocar uno mismo la asociación para que el AP emita ese M1.

Hay dos formas de provocar esa asociación. Las dos terminan igual: `hcxpcapngtool` extrae el PMKID (línea `WPA*01`) y `hashcat -m 22000` lo crackea.

```
# Localizar el objetivo
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon --band g       # he_pmkid_nc: CH 4, sin estaciones
BSSID=<BSSID_PMKID_NC>
```

Método A: hcxdumptool (ataque *clientless* "real")

hcxdumptool se hace pasar por un cliente con una MAC inventada, se autentica y se asocia con el AP, y guarda el M1 que este le envía.

```
# Interfaz limpia: hcxdumptool la pone en monitor él solo
sudo hcxdumptool -i wlan1 -c 4a -w pmkid_nc.pcapng --rds=1
#   -c 4a es imprescindible: por defecto solo salta por los canales 1, 6 y 11
#   (la "a" indica la banda de 2,4 GHz). Sintaxis de la 6.3 y posteriores.
#   Esperar a que la fila de he_pmkid_nc muestre "+" en la columna P y pulsar
Ctrl+C.

hcxpcapngtool -o pmkid_nc.22000 pmkid_nc.pcapng   # debe escribir una línea WPA*01
hashcat -m 22000 pmkid_nc.22000 /usr/share/wordlists/rockyou.txt
hashcat -m 22000 pmkid_nc.22000 --show
```

En la pantalla de hcxdumptool, las columnas `E A 1 2 3 P` de cada AP se marcan con `+`. Lo normal en este escenario es ver `+` en `E`, `A`, `1` y `P`, y nada en `2` ni en `3` (no hay clientes). Si `A` se queda en blanco, la asociación no se completa (ver más abajo: hostapd sin recompilar). Si `1` tiene `+` pero `P` no, el AP envía el M1 sin PMKID.

Requiere el hostapd recompilado con el parche del ACK. Si `/usr/local/sbin/hostapd-pmkid` es anterior a ese parche:

```
sudo ./limpia.sh
sudo ./compila_hostapd_pmkid.sh --force    # deben salir DOS "[OK] Parche
aplicado"
sudo ./despliegue.sh
```

Método B: `wpa_supplicant` con una contraseña falsa

El atacante se asocia como un cliente normal, con una contraseña cualquiera. El AP envía el M1 con el PMKID **antes** de comprobar nada. No necesita el hostapd recompilado con el parche del ACK.

```
# wlan1 en modo managed (si hcxdumptool la dejó en monitor)
sudo ip link set wlan1 down; sudo iw dev wlan1 set type managed; sudo ip link set
wlan1 up

# Terminal 1: capturar en el canal 4
sudo airodump-ng -c 4 --bssid $BSSID -w pmkid_nc wlan0mon

# Terminal 2: asociarse con una contraseña falsa
wpa_passphrase he_pmkid_nc contrasenafalsa > falso.conf
sudo wpa_supplicant -D nl80211 -i wlan1 -c falso.conf
#   ~10 s: saldrán errores de 4-way handshake (clave incorrecta). Ctrl+C

# Extraer SOLO el PMKID y crackear
hcxpcapngtool -o todo.22000 pmkid_nc-01.cap
grep 'WPA\*01' todo.22000 > pmkid_nc.22000
hashcat -m 22000 pmkid_nc.22000 /usr/share/wordlists/rockyou.txt
```

El `grep` es necesario: la captura también contiene el M2 que ha enviado `wpa_supplicant`, calculado con la contraseña falsa. hcxpcapngtool lo saca como `WPA*02`, y esa línea no tiene sentido crackearla.

Resultado esperado (ambos métodos): `football`.

Por qué hubo que recompilar hostapd para el método A

`compila_hostapd_pmkid.sh` aplica **dos** parches a hostapd 2.10. Los dos afectan solo al binario `hostapd-pmkid` (escenarios `he_pmkid` y `he_pmkid_nc`); el resto de escenarios usa el hostapd del sistema.

1. **PMKID en el M1 con WPA-PSK** (`src/ap/wpa_auth.c`). El hostapd de serie solo incluye el PMKID en el M1 con 802.1X, SAE u OWE. Sin este parche no hay PMKID que capturar con ningún método. Reproduce el comportamiento de muchos routers comerciales.
2. **No exigir el ACK de la *Authentication/Association Response*** (`src/ap/ieee802_11.c`, `handle_auth_cb` y `handle_assoc_cb`). En 802.11 el receptor de cada trama unicast la confirma con un ACK, y hostapd descarta a la estación si no recibe el ACK de esas dos respuestas. En `mac80211_hwsim` solo envían ACK las interfaces "activas" (managed, AP...). La MAC inventada que inyecta hcxdumptool desde modo monitor no pertenece a ninguna, así que nadie confirma las respuestas del AP. hostapd da la asociación por fallida y **nunca envía el M1**.

Síntoma sin el parche 2: la captura de hcxdumptool solo tiene un beacon y una *probe response*, y hcxpcapngtool avisa de que faltan tramas de autenticación y asociación (`no hashes written`). Para comprobarlo en directo:

```
sudo iw dev wlan0mon set channel 4
sudo tcpdump -i wlan0mon -e -n "wlan addr3 $BSSID and not subtype beacon"
# Se ve Authentication/Association, pero ninguna trama EAPOL del AP
```

El parche 2 solo cambia qué hace hostapd cuando **falta** el ACK. Los clientes legítimos y el método B (`wlan1` en managed) sí envían el ACK, así que siguen funcionando igual. El único efecto secundario es que cada intento

de hcxdumptool deja una estación "fantasma" hasta que caduca por inactividad (unos 5 minutos).

Es la misma limitación que impide el fake auth (`aireplay-ng -1`) en el escenario WEP.

Comparación (respuesta a la pregunta de la guía del alumno):

	`he_pmkid`	`he_pmkid_nc`
Cliente conectado	Sí	No
¿Hace falta esperar/deauth a un cliente?	Opcional	**No**
Material obtenible	PMKID **y** handshake	Solo PMKID
Lección	Dos vías de ataque	El PMKID no depende de clientes

Fundamento técnico

- Se aplica la misma fórmula que en el escenario anterior: `PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)`. Con PSK, la `MAC_STA` puede ser **la del atacante**: el PMKID no depende de ningún cliente legítimo. Cualquiera que se asocie recibe un M1 con un PMKID válido para su propia MAC.
- **Qué se explota.** El AP envía el M1 **antes** de que el cliente haya demostrado conocer la clave: el atacante no necesita la contraseña para provocarlo. En el 4-way, quien demuestra conocer la clave es el cliente, con el MIC del M2. Pero el PMKID ya va en el M1, que el AP envía sin verificación previa. Por eso el ataque es *clientless* y más sigiloso: no hay deauth ni hay que esperar a nadie.
  - Con hcxdumptool (método A) el atacante ni siquiera envía un M2.
  - Con `wpa_supplicant` (método B) se envía un M2 con una contraseña falsa, pero da igual: el PMKID del M1 ya está capturado.
- **Por qué sirve para crackear.** Todo lo que entra en la fórmula salvo la PMK es público: "PMK Name" es una constante, la MAC del AP sale en el beacon y la MAC de la estación la elige el atacante. Para cada contraseña candidata: `PMK = PBKDF2(candidata, SSID, 4096)` → `HMAC-SHA1-128(PMK, …)`. Si el resultado coincide con el PMKID capturado, la contraseña es esa. Es un ataque offline de diccionario: solo caen contraseñas débiles.
- **Contraste con el handshake clásico.** Sin clientes, el ataque al 4-way es imposible, porque nadie genera el M2 con el MIC. Con PMKID, un AP vacío sigue siendo atacable.
- **Mitigación:** igual que en WPA2-PSK (contraseña robusta, WPA3-SAE) y desactivar la caché de PMK / el PMKID en PSK si el fabricante lo permite.

---

6. `he_enterprise` — WPA2-EAP (PEAP/MSCHAPv2)

Hay un cliente conectado con PEAP/MSCHAPv2. El objetivo es **pasivo**: sacar de la captura la identidad del usuario, el método EAP y el certificado del servidor. El vector de **Evil Twin / RADIUS falso** (`hostapd-wpe`) se comenta en clase, pero no se ejecuta en este escenario.

Cómo está montado el escenario

- **El propio hostapd es el servidor de autenticación** (`eap_server=1`). No hay RADIUS externo. Usa un certificado autofirmado (`CN=radius.domain.lab`), que `despliegue.sh` genera la primera vez en `/opt/wifi-lab/enterprise/certs/`.

- **Sin caché de PMKSA** (`disable_pmksa_caching=1`). Cada conexión repite el EAP completo. Con la caché, el cliente reconectaría solo con el PMKID y no se vería ningún intercambio EAP.
- **El cliente se reasocia cada 30 s** (`wpa_cli reassociate`). Así la autenticación se repite sola y basta con escuchar: no hace falta deauth.
- **El cliente no valida el certificado** (no tiene `ca_cert`). Es la debilidad que se comenta en el fundamento técnico.

Al desplegar debe salir `[✓] Cliente he_enterprise autenticado (PEAP/MSCHAPv2)`.

a) Capturar

```
sudo airmon-ng start wlan0 3
sudo airodump-ng -c 3 --bssid <BSSID_ENT> -w cap_ent wlan0mon
# Esperar ~1 min (al menos una reasociación) y Ctrl+C
```

b) Identidad y método EAP

```
tshark -r cap_ent-01.cap -Y eap -T fields -e eap.code -e eap.type -e eap.identity
```

Se lee así:

`eap.code`	`eap.type`	**Significado**
1 (Request)	1	El AP pide la identidad
2 (Response)	1	El cliente responde con `student_user@domain.lab` **en claro**
1 (Request)	25	El servidor propone PEAP (*PEAP Start*)
1 / 2	25	Túnel TLS: a partir de aquí todo va cifrado
3 (Success)		Autenticación correcta; después viene el 4-way

c) Certificado del servidor

Con TLS 1.2, el certificado del servidor viaja en claro dentro de PEAP:

```
tshark -r cap_ent-01.cap -Y "tls.handshake.type == 11" -V | grep -i "id-at-
commonName"
```

Debe aparecer `radius.domain.lab`. Es lo que debería comprobar el cliente, y lo que imitaría un AP falso. Si la línea no sale, la sesión ha usado TLS 1.3, que cifra el certificado. No afecta al resultado de la práctica.

Resultado esperado: identidad `student_user@domain.lab` (viaja en claro en la fase externa), tipo EAP 25 = PEAP y certificado `CN=radius.domain.lab` autofirmado.

Si no aparece ningún paquete EAP: la captura empezó después de una reasociación y se cortó antes de la siguiente. Hay que esperar más. También puede que el cliente no se haya autenticado al desplegar. Para comprobarlo:

```
sudo ip netns exec ns_cli_ent wpa_cli -p /var/run/wpa_supplicant_ent -i
wlan_cli_ent status | grep -E "wpa_state|EAP"
# wpa_state=COMPLETED  /  EAP state=SUCCESS  /  selectedMethod=25 (EAP-PEAP)
```

Fundamento técnico

- **No hay una clave compartida que crackear.** En 802.1X cada usuario se autentica con sus credenciales contra un servidor de autenticación (normalmente RADIUS; en el lab, el EAP integrado de hostapd). La PMK se deriva de la sesión TLS (MSK) y es distinta para cada usuario y conexión. Capturar el handshake 4-way no sirve: la PMK no sale de ninguna contraseña común.
- **Fuga 1: la identidad externa.** El `EAP-Response/Identity` se envía **antes** de montar el túnel TLS, así que el usuario viaja en claro. Es lo que se obtiene en este lab. Mitigación: identidad externa anónima (`anonymous@dominio`).
- **Fuga 2: PEAP depende de que el cliente valide el certificado del servidor.** PEAP monta un túnel TLS autenticado solo con el certificado del RADIUS, y dentro va MSCHAPv2. Si el cliente no valida el certificado (o el usuario acepta el aviso), un **Evil Twin** con `hostapd-wpe` (RADIUS falso) completa el TLS con su propio certificado, y el cliente le entrega el reto y la respuesta MSCHAPv2.
- **Debilidad de MSCHAPv2.** Hash NT = MD4(UTF-16LE(contraseña)). La respuesta se calcula cifrando con DES el reto de 8 bytes con tres trozos de 7 bytes del hash NT; el tercero solo tiene 2 bytes reales. Esto permite:
  - un ataque de diccionario offline (`asleap`, `hashcat -m 5500`);
  - reducirlo a **un solo DES** (2^56) y recuperar el hash NT sea cual sea la contraseña. Ese hash ya sirve para *pass-the-hash*.
- **Mitigación:** forzar la validación del certificado en los clientes (CA fijada y nombre del servidor), identidad anónima y, mejor aún, **EAP-TLS** con certificado de cliente (sin contraseña que robar).

Mantenimiento: cambiar las claves

- **WPA2:** las claves se guardan como PMK (64 hex) en `despliegue.sh`, en el `hostapd.conf` y en el `wpa.conf` de cada escenario. La PMK depende del SSID, así que se calcula para cada red:

  ```
  ./calculaPMK.py he_wpa2 <nueva_clave>
  ```

  Usa palabras de `rockyou.txt` de 8 a 63 caracteres.

- **WEP:** `wep_key0` en hex. Solo admite 5 o 13 caracteres ASCII (10 o 26 hex):

  ```
  printf 'ilovemyfamily' | od -An -tx1 | tr -d ' \n'
  ```

Tras cambiar una clave, actualiza la tabla de soluciones de esta guía.

Verificación y apagado

```
# Ver APs y namespaces activos
ip netns list
sudo ip netns exec ns_ap_pmkid_nc iw dev

# Apagar todo el laboratorio
sudo ./limpia.sh
```

Herramientas usadas

- **aircrack-ng suite** (`airmon-ng`, `airodump-ng`, `aireplay-ng`, `aircrack-ng`)
- **hcxdumptool / hcxpcapngtool / hcxhashtool** (captura, conversión y filtrado de PMKID/handshake)
- **hashcat** (modo `22000` = WPA-PBKDF2-PMKID+EAPOL), en CPU mediante PoCL
- **tshark** (análisis de capturas)